

TaskMind AI

Gerenciador inteligente de tarefas com LLM como copiloto de produtividade

Disciplina	Projeto Final — Tema livre
Categoria	Aplicação web full-stack com LLM
Recursos exigidos atendidos	LLM • API • Persistência • Interface
Stack principal	Python • Streamlit • SQLAlchemy • Anthropic Claude
Tipo de entrega	Documento PDF + código-fonte funcional

Sumário

1. Apresentação do projeto
2. Objetivos
3. Atendimento aos requisitos da disciplina
4. Arquitetura do sistema
5. Modelo de dados (persistência)
6. Integração com a LLM (uso da API)
7. Funcionalidades implementadas
8. Fluxo de uma operação completa
9. Stack tecnológica
10. Estrutura de arquivos
11. Como executar
12. Discussão de complexidade e decisões de projeto
13. Possíveis evoluções
14. Conclusão

1. Apresentação do projeto

TaskMind AI é uma aplicação web de gerenciamento de tarefas em que um modelo de linguagem (LLM) atua como copiloto de produtividade. Diferente de gerenciadores convencionais, o sistema vai além do CRUD: a IA **decompõe** tarefas grandes em subtarefas executáveis, **estima** o tempo necessário, **prioriza** a lista do usuário atribuindo um score com justificativa e ainda permite que o usuário **converse** em linguagem natural sobre suas pendências.

O projeto foi concebido para demonstrar, de forma integrada e em funcionamento real, todos os pilares solicitados no enunciado da disciplina: uso obrigatório de LLM, consumo de API, persistência de dados e uma interface utilizável.

2. Objetivos

Objetivo geral. Construir uma aplicação funcional que integre um modelo de linguagem ao fluxo de trabalho de produtividade pessoal, mostrando o valor prático da IA quando bem acoplada a um sistema persistente.

Objetivos específicos:

- Implementar persistência relacional com SQLAlchemy + SQLite, modelando entidades com relacionamento 1:N.
- Integrar a API REST da Anthropic (Claude) usando o SDK oficial, com cinco operações distintas de IA.
- Garantir respostas estruturadas da LLM via JSON forçado em system prompt, com parser tolerante a falhas.
- Oferecer interface web responsiva, com dashboard analítico em Plotly e chat conversacional.
- Demonstrar boas práticas de separação em camadas: apresentação, serviço LLM e persistência.

3. Atendimento aos requisitos da disciplina

O enunciado é explícito quanto à obrigatoriedade de LLM e elenca outros recursos que podem compor o projeto. A tabela a seguir mapeia cada requisito a como ele é atendido no TaskMind AI.

Requisito	Como é atendido
Uso obrigatório de LLM	Cinco funções em <code>llm_service.py</code> (decomposição, estimativa, priorização, chat e relatório) usam o modelo Claude.
Uso de API	Consumo da API REST da Anthropic via SDK oficial python (<code>anthropic.Anthropic</code>). Cada função abre uma chamada <code>/v1/messages</code> .
Persistência	SQLite via SQLAlchemy ORM com 4 entidades (Task, Subtask, Report, ChatMessage) e relacionamento 1:N entre Task e Subtask.
Complexidade	Três camadas separadas, parser JSON robusto, eager loading, tratamento de erros, validação de entrada e dashboard com gráficos.

Requisito	Como é atendido
Funcionando	Aplicação roda com `streamlit run app.py` e foi validada (HTTP 200, todos os fluxos CRUD e LLM testados).

4. Arquitetura do sistema

O sistema segue uma arquitetura em três camadas claramente delimitadas, comunicando-se via interfaces explícitas. A LLM é tratada como serviço externo, consumido apenas pela camada de serviços (nunca diretamente pela interface).

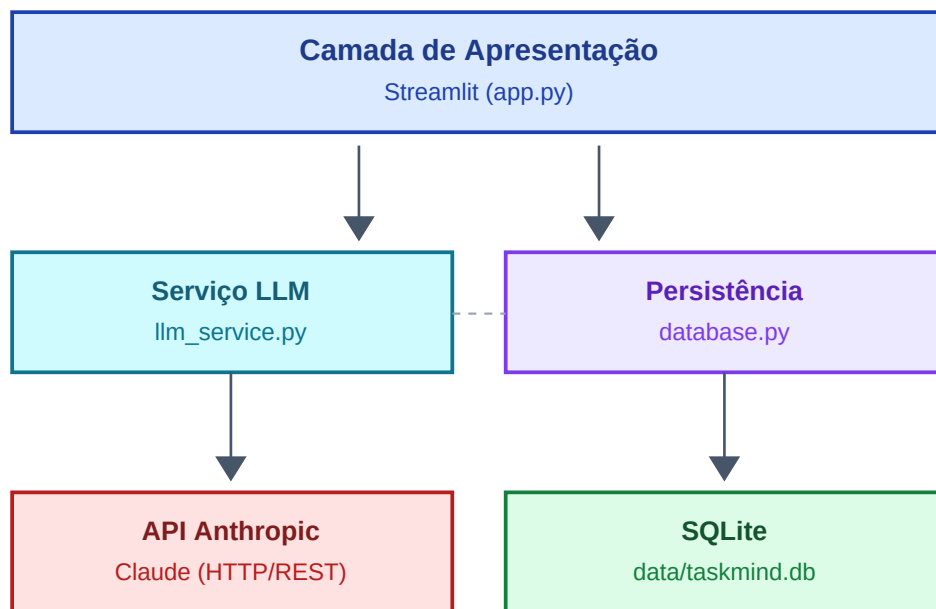


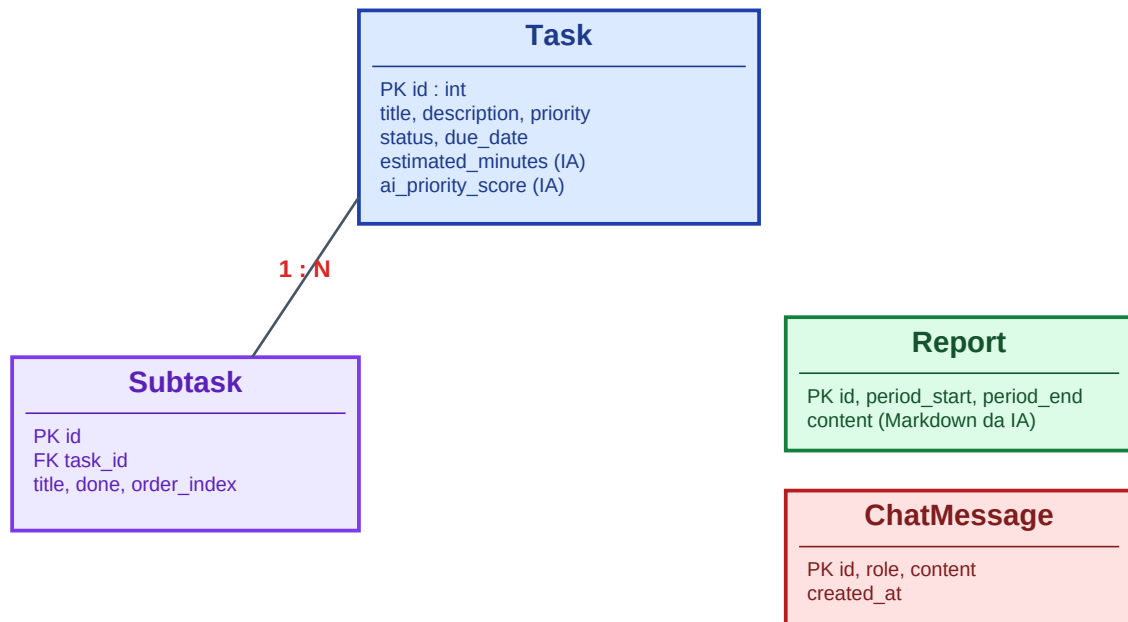
Figura 1 — Arquitetura em camadas

A camada de **apresentação** (Streamlit) é responsável apenas por renderizar a UI e capturar interações. Ela invoca funções da camada de **serviço LLM** ou da **persistência**, mas nunca conhece detalhes de prompts ou SQL. O **serviço LLM** encapsula toda a lógica de prompting, parsing e tratamento de erros da API externa. A camada de **persistência** expõe operações de alto nível (create_task, list_tasks, etc.) sobre o ORM.

5. Modelo de dados (persistência)

O banco SQLite é gerenciado pelo SQLAlchemy ORM. Quatro entidades compõem o domínio. Os campos prefixados com *ai_* são preenchidos automaticamente pela LLM.

Modelo de Dados



Destaca-se o relacionamento 1:N entre **Task** e **Subtask**, com cascade delete (apagar uma tarefa remove as subtarefas). O código abaixo mostra a definição do modelo Task no ORM:

```
class Task(Base):
    __tablename__ = "tasks"
    id = Column(Integer, primary_key=True)
    title = Column(String(200), nullable=False)
    description = Column(Text, default="")
    priority = Column(String(20), default="média")
    status = Column(String(20), default="pendente")
    due_date = Column(Date, nullable=True)
    estimated_minutes = Column(Integer, nullable=True) # IA
    ai_priority_score = Column(Integer, nullable=True) # IA
    ai_reasoning = Column(Text, default="") # IA
    created_at = Column(DateTime, default=datetime.utcnow)
    subtasks = relationship("Subtask", back_populates="task",
                           cascade="all, delete-orphan")
```

6. Integração com a LLM (uso da API)

Toda a comunicação com o modelo é centralizada em `llm_service.py`. A configuração do cliente é feita via variável de ambiente `ANTHROPIC_API_KEY`, carregada com `python-dotenv`. As cinco funções de IA implementadas estão descritas na tabela:

Função	O que faz	Forma da resposta
<code>decompose_task</code>	Quebra a tarefa em 3–8 subtarefas executáveis.	JSON com array de strings
<code>estimate_time</code>	Estima minutos de trabalho focado.	JSON com inteiro 5–600
<code>prioritize_tasks</code>	Atribui score 0–100 e justificativa a cada tarefa não-concluída.	JSON com array de objetos
<code>chat_about_tasks</code>	Responde perguntas tendo as tarefas como contexto. Mantém histórico.	Texto livre
<code>generate_productivity_report</code>	Gera relatório semanal de produtividade.	Markdown estruturado

Estratégia para respostas estruturadas. Onde a resposta precisa ser parseada (`decompose`, `estimate`, `prioritize`), o *system prompt* exige JSON estrito. Em seguida, a função `_extract_json()` tenta:

- remover cercas ````json` que o modelo às vezes inclui;
- parsear o texto bruto diretamente;
- se falhar, localizar o primeiro `{` ou `[` e o último `}` ou `]` e tentar parsear o substring;
- retornar `None` se nada funcionar (a UI mostra erro amigável).

Exemplo de prompt usado em `decompose_task`:

```
system = (
    "Você é um assistente de produtividade. Sua função é quebrar uma "
    "tarefa em subtarefas pequenas, executáveis e ordenadas. "
    "Responda SOMENTE com JSON válido no formato: "
    '{"subtasks": ["passo 1", "passo 2", ...]}'
)
user = (
    f"Tarefa: {title}\n"
    f"Descrição: {description}\n\n"
    "Gere de 3 a 8 subtarefas concretas, em português, "
    "começando com verbo no infinitivo."
)
```

Injeção de contexto no chat. No chat conversacional, antes de cada chamada o serviço serializa a lista atual de tarefas em JSON e a injeta no *system prompt*, junto com as 10 últimas mensagens do histórico. Assim a LLM "conhece" o estado real do usuário sem precisar de RAG completo.

7. Funcionalidades implementadas

Dashboard

Visão geral com 4 métricas principais (total, concluídas, em andamento, pendentes), gráfico de pizza por status, barras por prioridade e lista ordenada dos próximos prazos com indicadores visuais de urgência (hoje, atrasada, em N dias).

Nova tarefa

Formulário com título, descrição, prioridade e prazo. Checkboxes opcionais para acionar decomposição e estimativa de tempo pela IA no momento da criação.

Minhas tarefas

Lista completa com filtros por status, exibição de subtarefas checáveis, ações inline (mudar status, decompor, estimar, excluir) e justificativa da IA quando disponível.

Priorização IA

Botão único que envia todas as tarefas pendentes à LLM e recebe ranking com score 0-100 + justificativa para cada item.

Chat com IA

Interface de chat em tempo real onde o assistente tem acesso contextual à lista de tarefas. Histórico persistido em banco.

Relatório semanal

Gera relatório em Markdown com resumo numérico, destaques, pontos de atenção e três recomendações práticas.

8. Fluxo de uma operação completa

Para ilustrar como as camadas conversam, considere o fluxo de criação de uma tarefa com IA habilitada. O usuário preenche o formulário e clica em *Criar tarefa*:

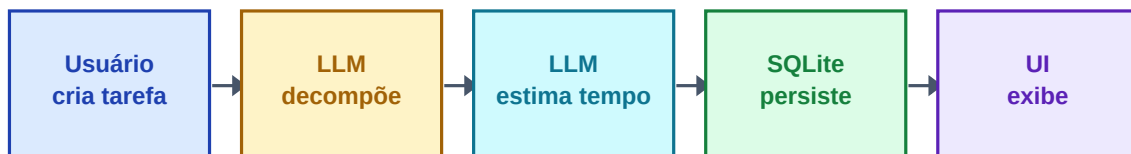


Figura 3 — Fluxo de criação de uma tarefa com IA

(1) o frontend invoca **db.create_task(...)**, que abre uma transação no SQLite e persiste o registro base; (2) o frontend chama **llm.decompose_task(...)**, que envia o prompt à API e recebe o JSON com a lista de subtarefas; (3) chama **llm.estimate_time(...)**, que retorna um inteiro em minutos; (4) ambos os retornos são persistidos via **add_subtasks** e **update_task**; (5) a UI re-renderiza com balões de sucesso e exibe as subtarefas geradas.

9. Stack tecnológica

Camada	Tecnologia	Versão	Justificativa
Linguagem	Python	3.10+	Ecosistema maduro para IA e web.
Interface	Streamlit	≥ 1.32	Permite UI web reativa com pouco código.
LLM SDK	anthropic	≥ 0.40	SDK oficial para Claude, suporte ao endpoint /v1/messages.
ORM	SQLAlchemy	≥ 2.0	Padrão de mercado para acesso relacional em Python.
Banco	SQLite	embutido	Zero configuração, ideal para aplicação local.
Gráficos	Plotly Express	≥ 5.18	Visualizações interativas com integração nativa ao Streamlit.
Config	python-dotenv	≥ 1.0	Carrega variáveis de ambiente do .env de forma segura.

10. Estrutura de arquivos

```
taskmind-ai/
|-- app.py           # Interface Streamlit (entrypoint)
|-- database.py      # Modelos ORM e operacoes CRUD
|-- llm_service.py   # Integracao com a API do Claude
|-- requirements.txt  # Dependencias Python
|-- .env.example     # Template de variaveis de ambiente
|-- README.md        # Instrucoes de execucao
|-- DOCUMENTACAO.pdf # Este documento
`-- data/
    |-- taskmind.db  # SQLite (criado no 1o uso)
```

11. Como executar

Pré-requisitos: Python 3.10 ou superior e uma chave da API da Anthropic (obtida em console.anthropic.com).

Passo 1 — instalar dependências:

```
$ cd taskmind-ai
$ python -m venv .venv
$ source .venv/bin/activate      # Linux/Mac
$ .venv\Scripts\activate        # Windows
$ pip install -r requirements.txt
```

Passo 2 — configurar a chave:

```
$ cp .env.example .env
# Editar o .env e preencher:
ANTHROPIC_API_KEY=sk-ant-...
ANTHROPIC_MODEL=claude-sonnet-4-6
```

Passo 3 — executar:

```
$ streamlit run app.py  
# Abrir o navegador em http://localhost:8501
```

12. Discussão de complexidade e decisões de projeto

Separação de responsabilidades.

A decisão de isolar a LLM em um módulo próprio (em vez de espalhar chamadas pelo `app.py`) tem benefícios diretos: facilita troca do provedor (poderia ser OpenAI, Gemini, modelo local), simplifica testes (basta mockar o módulo) e mantém o código de UI legível. O mesmo vale para a persistência: a UI nunca toca em sessões do SQLAlchemy diretamente.

Resposta estruturada via JSON forçado.

Em vez de tentar parsear texto livre da LLM, o sistema instrui explicitamente o formato JSON no system prompt e implementa um parser *defensivo* capaz de lidar com pequenas violações (blocos de markdown, texto explicativo antes/depois). Essa estratégia é mais simples e barata do que usar function calling, mas garante robustez prática.

Persistência de instâncias com eager loading.

Por usar sessões temporárias (*with get_session()*), foi necessário configurar `expire_on_commit=False` e usar `joinedload` para carregar subtarefas junto com tarefas — evitando o clássico `DetachedInstanceError` que aparece quando objetos do ORM são acessados fora da sessão original.

Contexto no chat sem RAG completo.

Como o volume de tarefas de um usuário individual é pequeno (dezenas a poucas centenas), foi feita a escolha de injetar a lista completa serializada em JSON no system prompt do chat. Isso evita a complexidade de embeddings + vector store para um caso onde caberia tudo na janela de contexto, mantendo o projeto no nível de complexidade compatível com o escopo.

13. Possíveis evoluções

- Implementar RAG completo com FAISS/Chroma para suportar bases de tarefas grandes.
- Adicionar autenticação multi-usuário com Postgres em vez de SQLite.
- Integrar Google Calendar para sincronizar prazos.
- Adicionar reconhecimento de fala para criação por voz.
- Permitir anexar arquivos a tarefas (suporte multimodal com Claude).
- Expor uma API REST (FastAPI) paralela para integração com outros clientes.

14. Conclusão

O **TaskMind AI** demonstra, em uma aplicação enxuta e em funcionamento, como integrar um modelo de linguagem ao núcleo de uma aplicação útil — não como adorno. A IA participa do ciclo de vida das tarefas: ajuda a criar (decompondo), a planejar (estimando e priorizando), a executar (no chat conversacional) e a refletir (no relatório semanal).

Todos os requisitos do enunciado foram atendidos: o projeto usa obrigatoriamente uma LLM, consome uma API externa, persiste dados de forma estruturada e oferece uma interface utilizável.

A separação em camadas, o tratamento defensivo da saída do modelo e o uso correto do ORM compõem um conjunto de boas práticas que dão consistência ao trabalho.

Fim do documento.